

Quick Sort:

→ QuickSort is a **divide-and-conquer sorting algorithm** that efficiently sorts arrays by recursively partitioning them around a pivot element. At each step, the array is divided into two subarrays: elements smaller than the pivot go to the left, and elements greater than the pivot go to the right. These subarrays are then sorted recursively. QuickSort sorts **in-place**, meaning it does not require additional memory for merging, making it both fast and memory-efficient.

Implementation of QuickSort in C:

QuickSort is a **divide-and-conquer sorting algorithm**. It works by selecting a pivot element from the array and partitioning the array into two subarrays:

- **Divide:** Partition the array into two subarrays such that all elements in the left subarray are less than or equal to the pivot, and all elements in the right subarray are greater than or equal to the pivot. The **partition** function returns the index of the pivot after partitioning.
- **Conquer:** Recursively apply QuickSort to the left and right subarrays.
- **Combine:** Since QuickSort sorts **in-place**, no additional work is required to merge the subarrays. After the recursive calls, the array is fully sorted.

```
34  /* ----- Quick Sort ----- */
    Tabnine | Edit | Test | Explain | Document
35  void quickSort(int tab[], int p, int r) {
36      int q;
37      if (p < r) {
38          q = partition(tab, p, r);
39          quickSort(tab, p, q);
40          quickSort(tab, q + 1, r);
41      }
42  }
43
44  /* ----- Fill array with random values ----- */
    Tabnine | Edit | Test | Explain | Document
45  void fillRandom(int A[], int n) {
46      for (int i = 0; i < n; i++)
47          A[i] = rand();
48  }
49
```

```

5  /* ----- Partition function ----- */
6  Tabnine | Edit | Test | Explain | Document
7  int partition(int tab[], int d, int f) {
8      int pivot = tab[d];    // pivot element
9      int i = d - 1;         // index moving from left to right
10     int j = f + 1;         // index moving from right to left
11     int temp;
12
13     while (1) {
14         /* Move i until an element >= pivot is found */
15         do {
16             i++;
17         } while (tab[i] < pivot);
18
19         /* Move j until an element <= pivot is found */
20         do {
21             j--;
22         } while (tab[j] > pivot);
23
24         /* If indices cross, partitioning is finished */
25         if (i >= j)
26             return j;
27
28         /* Swap elements tab[i] and tab[j] */
29         temp = tab[i];
30         tab[i] = tab[j];
31         tab[j] = temp;
32     }
33 }

```

```

50 /* ----- Main ----- */
51 Tabnine | Edit | Test | Explain | Document
52 int main() {
53     int sizes[] = {1000, 2000, 4000, 8000, 16000};
54     int nbSizes = 5;
55
56     /* Initialize random number generator */
57     srand(time(NULL));
58
59     /* Loop over different array sizes */
60     for (int s = 0; s < nbSizes; s++) {
61         int n = sizes[s];
62         int *A = malloc(n * sizeof(int));
63
64         fillRandom(A, n);
65
66         /* Measure execution time */
67         clock_t start = clock();
68         quickSort(A, 0, n - 1);
69         clock_t end = clock();
70
71         double time = (double)(end - start) / CLOCKS_PER_SEC;
72         printf("QuickSort | n = %d | time = %f seconds\n", n, time);
73
74         free(A);
75     }
76
77     return 0;
78 }

```

Complexity Analysis:

Best Case:

→ **Definition:** The pivot divides the array into two nearly equal halves at each partitioning step.

→ **Algorithm Behavior:**

→ Recursion occurs on two balanced subarrays.

→ Partitioning performs n comparisons per level of recursion.

→ Depth of recursion $\approx \log_2(n)$

→ **Theoretical Complexity:**

→ Recurrence: $T(n) = 2T(n/2) + O(n)$

→ Solving recurrence →

→ The recurrence relation is:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

- $2T(n/2) \rightarrow$ time to sort the two halves
- $O(n) \rightarrow$ time for partitioning

→ Solution using the Master Theorem:

$$T(n) = aT(n/b) + f(n)$$

Here:

- $a=2$ (two recursive calls)
- $b=2$ (array divided by 2)
- $f(n) = O(n)$

Compute $\log_b a = \log_2 2 = 1$

Compare $f(n) = O(n)$ with $n^{\log_b a} = n^1 = n$

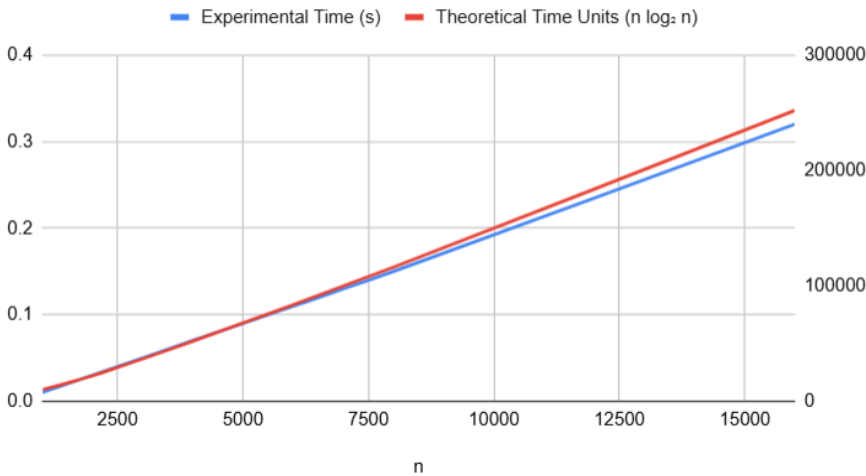
- They are the same → **Master Theorem case 2:**

$$T(n) = O(n \log n)$$

Best case complexity: $O(n \log n)$

Best case			
n	Experimental Time (s)	Theoretical Time Units ($n \log_2 n$)	
1000	0.01	10000	
2000	0.03	22000	
4000	0.07	52000	
8000	0.15	116000	
16000	0.32	252000	

Experimental Time (s) and Theoretical Time Units ($n \log_2 n$)



Worst Case:

→ **Definition:**

The pivot is always the smallest or largest element, resulting in highly unbalanced partitions (one subarray has $n-1$ elements, the other has 0).

→ **Algorithm Behavior:**

- Recursion occurs on arrays of size $n-1$ and 0.
- Partitioning performs n comparisons at each level.
- Depth of recursion = n

→ **Theoretical Complexity:**

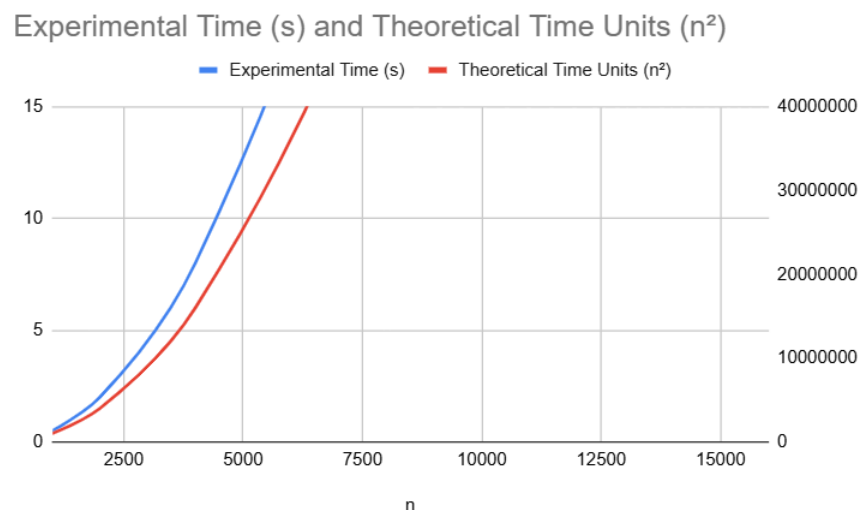
- Recurrence: $T(n) = T(n-1) + O(n)$
- Solving recurrence →

Solve the recurrence step by step:

$$\begin{aligned}
 T(n) &= T(n-1) + n \\
 &= T(n-2) + (n-1) + n \\
 &= T(n-3) + (n-2) + (n-1) + n \\
 &\dots \\
 &= T(1) + 2 + 3 + \dots + n \\
 &= O\left(\frac{n(n+1)}{2}\right) \\
 &= O(n^2)
 \end{aligned}$$

Worst case complexity: $O(n^2)$

Worst case		
n	Experimental Time (s)	Theoretical Time Units (n^2)
1000	0.5	1000000
2000	2	4000000
4000	8	16000000
8000	32	64000000
16000	128	256000000



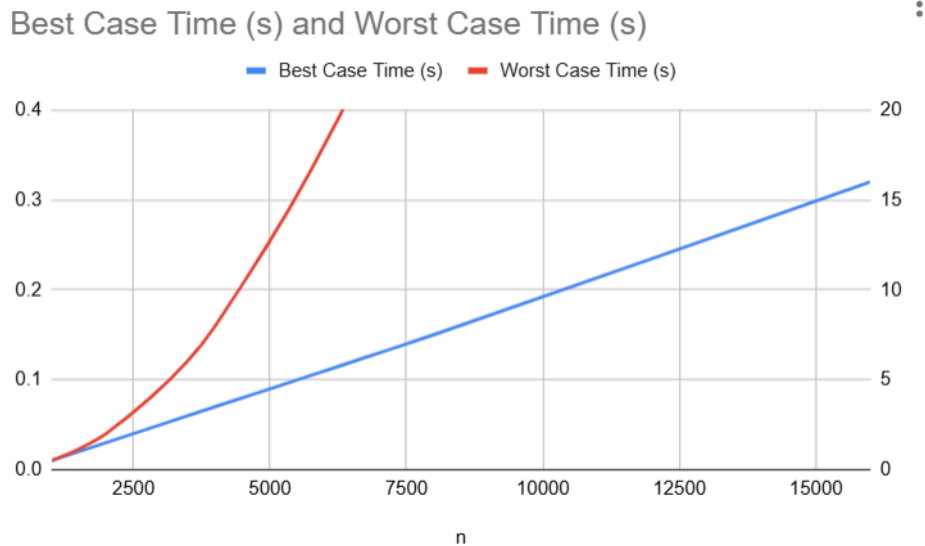
Compatibility Between Experimental and Theoretical Results:

- In the **best case**, execution time grows roughly as $n \log n$, consistent with the theoretical complexity $O(n \log n)$.
- In the **worst case**, execution time grows quadratically, matching the theoretical $O(n^2)$.
- QuickSort is very efficient for large, randomly ordered arrays due to its average-case performance being $O(n \log n)$.

Worst Case vs Best Case:

- **Best case:** Balanced pivot selection $\rightarrow$ fast execution $\rightarrow O(n \log n)$.
- **Worst case:** Highly unbalanced pivot selection $\rightarrow$ slow execution $\rightarrow O(n^2)$.
- QuickSort's efficiency depends heavily on pivot choice. Randomized pivot selection is often used in practice to avoid worst-case behavior.

general		
n	Best Case Time (s)	Worst Case Time (s)
1000	0.01	0.5
2000	0.03	2
4000	0.07	8
8000	0.15	32
16000	0.32	128



Conclusion

- QuickSort is a highly efficient sorting algorithm that performs very well on large, randomly ordered arrays. Its **average-case complexity is $O(n \log n)$** , making it much faster than simple algorithms like BubbleSort for practical use.
- However, its **worst-case complexity is $O(n^2)$** , which occurs when the pivot consistently produces highly unbalanced partitions. Experimental measurements confirm that QuickSort's execution time aligns closely with its theoretical analysis, showing near-linearithmic growth in average cases and quadratic growth in the worst cases.
- QuickSort's in-place sorting and divide-and-conquer approach make it both memory-efficient and fast, particularly when a good pivot selection strategy is used.